

Task 7 — Real-World Applications and

SDG Reflection

Part - A : Timer-Based Applications in Modern Embedded Systems

1. RTOS (Real Time Operating System) : Real-time control systems running on modern industrial control equipment need many types of high-precision timing services, a design method of timer based on FreeRTOS real-time embedded operating system is proposed. RTOS-based embedded systems rely heavily on timer peripherals for scheduling, synchronization, and real-time control operations.

Timer Functions Used: Periodic timer interrupts, Delay generation, Combination of timer counting and task scheduling

Real time operating systems require very high timing accuracy because real-time tasks must execute exactly on schedule, communication protocols depend on deterministic timing, and industrial machines must remain synchronized.

If the timing was 10% off, real-time tasks would miss their deadlines and execute at incorrect times, communication protocols would become unreliable due to non-deterministic delays or data loss, and industrial machines would lose synchronization leading to incorrect operation, system instability and possible equipment failure.

2. Automotive Engine Control Unit (ECU) : Modern car engines rely heavily on timer peripherals inside the ECU for ignition timing and fuel injection.

Timer Functions Used : PWM, Precise Delay Generation, Frequency measurement

Ignition timing accuracy is often required in the microsecond range because combustion events happen extremely quickly at high RPM.

If ignition or injection timing shifts by 10%, fuel burns inefficiently, engine knocking may occur, power drops, fuel consumption increases, engine overheating can happen, and emissions rise dramatically.

3. Drone Flight Controller System : Modern drones use embedded systems to stabilize flight, control motor speed, and maintain balance. Timer peripherals are extremely important in these systems.

Timer Functions Used : PWM (Pulse Width Modulation), Periodic timer interrupts, Frequency measurement from sensors

The timing error usually must remain within a few microseconds because all motors must rotate at carefully synchronized speeds, small timing differences change thrust balance, and stable flight depends on accurate motor timing.

If timing is 10% off then result will be drone instability, uncontrolled rotation and possibly crash

Part - B : WS2812B and Smart Lighting in Smart Cities

i) The WS2812B LED is commonly used in smart lighting systems because each LED can be individually controlled using PWM-based data encoding.

By adjusting the duty cycle (ON vs OFF time), the brightness of LEDs can be precisely controlled. Lower brightness means reduced power consumption because LEDs consume power proportional to their ON time.

Relation to SDG Goals:

SDG Goal 7 (Affordable and Clean Energy):

PWM-based dimming reduces electricity consumption, lowering operational costs and making lighting systems more energy-efficient and sustainable.

SDG Goal 11 (Sustainable Cities and Communities):

Smart dimming allows adaptive lighting in urban areas, reducing light pollution while maintaining safety, contributing to smarter and more sustainable city infrastructure.

ii) A distributed smart lighting system can be built using multiple STM32 microcontroller nodes connected via a CAN bus.

System Design: Each streetlight acts as a node. Nodes communicate sensor data (light intensity, motion detection). A central controller or peer nodes adjust brightness dynamically.

Operation:

- light sensors detect natural light levels.
- PIR or radar sensors detect pedestrian or vehicle presence.
- Based on inputs Lights brighten when motion is detected,Lights dim when no activity is present, System adapts in real time for energy saving

3. Firmware Design :

For a smart street lighting system expected to operate continuously for 10 years without maintenance, reliable and efficient firmware design is essential. An interrupt-driven architecture should be used for handling sensor events and CAN bus communication because it reduces unnecessary CPU activity and improves responsiveness. DMA (Direct Memory Access) should be used for continuous data transfer tasks, such as LED data transmission and communication buffering, since it reduces CPU load and increases system efficiency.

Low-power sleep modes are important to minimize energy consumption during idle periods. The STM32 controller can remain in sleep or standby mode and wake up only when motion is detected or a communication interrupt occurs. This significantly improves long-term energy efficiency.

Watchdog timers are also essential for system reliability. If the firmware becomes unresponsive due to software faults, memory corruption, or unexpected errors, the watchdog timer automatically resets the system and restores normal operation. This ensures stable long-term deployment without manual maintenance.

Part - C : Greenhouse Node Firmware Architecture

1. RGB Status LED (WS2812B)

Peripherals: TIM1 + DMA

The WS2812B LED requires highly accurate PWM timing for transmitting 24-bit RGB data. TIM1 is selected because it is an advanced-control timer capable of generating precise PWM waveforms.

DMA is used to transfer precomputed PWM duty-cycle values directly into the CCR1 register, minimizing CPU involvement.

System Status Indication : Green → Normal, Yellow → Warning, Red → Fault

Bare-Metal vs HAL: Bare-metal programming is preferred because WS2812B communication requires strict timing accuracy in the nanosecond range. HAL overhead may introduce timing jitter and corrupt LED data transmission.

2. Servo-Controlled Ventilation Flap

Timer Peripherals: TIM3 PWM Mode

The ventilation flap is controlled using a servo motor that requires:

- 50 Hz PWM frequency
- Pulse widths between 1 ms and 2 ms

TIM3 is suitable for generating stable PWM output signals for servo control.

Bare-Metal vs HAL: HAL is acceptable because servo control is not extremely timing-sensitive and updates occur infrequently. HAL simplifies PWM configuration and improves maintainability.

3. Temperature/Humidity Sensor Response-Time Measurement

Timer Peripheral: TIM2 Input Capture Mode

TIM2 is selected because it is a 32-bit timer capable of accurately measuring long sensor pulse durations without overflow.

The timer captures:

- Rising edge timestamp
- Falling edge timestamp

The sensor response time is determined from the difference between these timestamps.

Bare-Metal vs HAL : Bare-metal programming is preferred because direct register access minimizes interrupt latency and improves timing precision.

4. UART Telemetry Transmission

Timer+Peripheral: TIM6 + USART2

TIM6 generates a periodic interrupt every 1 second. A software counter increments in the interrupt service routine, and when the counter reaches 30:

- Sensor readings are collected
- UART transmission begins through USART2

Bare-Metal vs HAL: HAL is suitable because UART communication does not require extremely precise timing. HAL simplifies peripheral initialization and communication handling.

5. Low-Power Operation

the system is solar-powered, minimizing energy consumption is necessary for the microcontroller board.

The STM32F446 enters: Sleep mode and Stop mode

between telemetry cycles.

The system wakes only when: Timer interrupts occur and then Sensor activity is detected and UART transmission is required

6. Task Coordination Without RTOS

The system coordinates all operations using: Interrupt-driven architecture, Volatile software flags and Super-loop scheduling

instead of an RTOS.

Coordination Method

Each interrupt service routine: Sets a software flag and Returns immediately

The main loop continuously executes: WFI (Wait For Interrupt)

to place the CPU into low-power sleep mode.

When an interrupt occurs:

1. The CPU wakes up
2. Flags are checked in priority order
3. The highest-priority task is serviced
4. The system returns to sleep mode

Priority Order: Fault handling, Sensor measurement, UART transmission, LED update

This architecture minimizes CPU activity and reduces power consumption while maintaining reliable operation.

